

M1 Technical Briefing: High-Level Synthesis with Game Theoretic Scheduling

Sumon Boiddo

Student ID: 2220054

Hochschule Hamm-Lippstadt

Lippstadt, Germany

sumon.boiddo@stud.hshl.de

Abstract—[Milestone 1: Structured Technical Briefing. This document uses bullet points, formulas, pseudocode, and tables rather than prose, per the M1 format rules. It presents the core technical idea of using game theory for power-aware scheduling and binding in High-Level Synthesis (HLS): functional units act as players in a first-price sealed-bid auction, and the Nash equilibrium selects the lowest-power assignment. The supporting benchmark data referenced for context originates from Murugavel and Ranganathan [1]. AI usage disclosed in the accompanying protocol.]

I. PROBLEM STATEMENT AND MOTIVATION

A. High-Level Synthesis (HLS): Context

- **Definition:** HLS converts an algorithm-level description (C/C++) directly into a hardware design, instead of manual gate-level design [5].
- **Why HLS:** generating a hardware circuit from high-level code by hand is extremely difficult; HLS makes complex designs feasible.
- **Three steps of HLS** [4], [5]:
 - 1) **Scheduling:** decides which operation runs first — assigns the time / clock cycle.
 - 2) **Allocation:** decides how many resources (adders, multipliers, subtractors) are needed.
 - 3) **Binding:** decides which specific hardware unit performs which task.
- Scheduling is performed first and strongly affects power, speed, latency, and hardware usage.

B. Why Power Optimization Matters

- High power $\Rightarrow$ faster battery drain, device heating, higher cost, lower reliability.
- Reliability of circuits drops by about **50% for every 10°C** rise in temperature [3].
- Critical for **mobile devices, IoT devices, and sensor systems** — small batteries, strict power limits, cost-sensitive.
- Within HLS, **scheduling** and **binding** waste the most power if handled poorly:
 - Poor scheduling $\Rightarrow$ unnecessary switching activity $\Rightarrow$ higher power and latency.

- Wrong binding (e.g. wrong adder chosen) $\Rightarrow$ increased switching activity of that unit $\Rightarrow$ wasted power.
- Allocation imbalance: too many units $\Rightarrow$ more area/power; too few $\Rightarrow$ more latency.

C. The Problem This Work Addresses

- **Goal:** assign operations in HLS to hardware units so that the *least power* is wasted, without increasing chip area or causing large latency increases.
- **Older methods:** ILP, LP, simulated annealing, force-directed scheduling [2], [6], [7], [8].
- **Weakness of older methods:** complex, did not give the best results, and typically optimised *either* scheduling *or* binding — not both together.
- **Approach taken here:** model the assignment of operations to units as a game; each unit bids by its power cost, and the lowest-cost bid wins.

II. CORE TECHNICAL IDEA

A. Key Insight

- Every hardware unit (adder, multiplier, etc.) acts as a **player** in a game.
- Each player **bids** according to its own power cost.
- The player with the **lowest operation cost** gets that task.
- The stable outcome is found via the **Nash equilibrium**: no player can improve its result by unilaterally changing its decision.

B. Game-Theoretic Components

- 1) **Players:** the functional units (adders, multipliers, subtractors).
- 2) **Strategies:** which operation a unit chooses to perform.
- 3) **Bid:** the power cost a unit would incur to perform an operation.
- 4) **Auction type:** first-price sealed-bid — the lowest power bid wins.
- 5) **Solution concept:** Nash equilibrium — the stable set of bids.

III. FORMALISM

A. Nash Equilibrium Payoff, Eq. (1)

$$\bar{p} = \sum_{i=1}^N p_{(n_1^*, \dots, n_N^*)}^i \quad (1)$$

- Every hardware unit (adder, multiplier) acts as a player and makes a final decision about which operation to perform and what the power cost will be.
- When all players have made final decisions, the total cost of all players is summed to give the overall payoff [1].
- $n_1^*, \dots, n_N^*$: the equilibrium strategy of each of the N players.

B. Binding Cost Function, Eq. (2)

$$co(m, o) = P_{sw} - P_{inp} \quad (2)$$

- P_{sw} : power consumed due to **switching**.
- P_{inp} : power *reduced* due to common / neighbourhood inputs between operations.
- Lower final cost $\Rightarrow$ that hardware unit is the better choice for the operation [1].

C. Optimal Assignment, Eq. (3)

$$CO(A^*) = \min_{A \subseteq S} CO(A) \quad (3)$$

- A : a possible module–operation assignment (which operation goes to which adder/multiplier).
- $CO(A)$: the total power cost of assignment A .
- The system searches all possible assignments and chooses the one consuming the least power [1].

IV. METHODOLOGY AND ALGORITHM

A. From HLS Problem to Game Problem

- Identify which operations exist and which functional units are available (adders, multipliers).
- Treat **power, delay, and area** as cost / resources in the game.
- Begin the bidding: each functional unit bids using its power cost.
- Example: if three adders bid 8, 5, and 3 power units, the adder bidding 3 wins the operation (auction mechanism).

B. Level-by-Level Scheduling Flow

C. Scheduling Procedure (level by level)

- 1) Examine the dataflow graph (DFG).
- 2) Operations ready at the beginning form the **top level**.
- 3) Functional units bid for these operations; the Nash equilibrium decides the winner.
- 4) Schedule the winning operations at that level.
- 5) Remove scheduled operations from the graph; the next level becomes ready.
- 6) Repeat until all operations are scheduled.

Output: a complete schedule defining which task happens when, which unit performs it, arranged for minimum power.

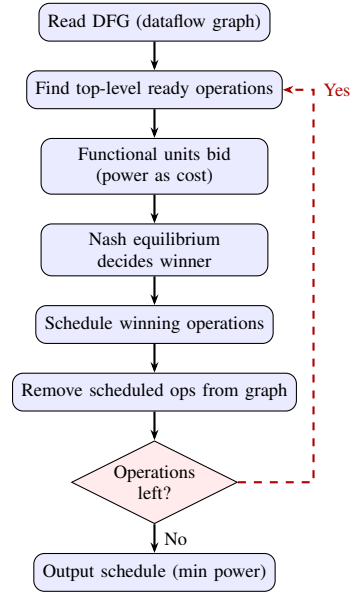


Fig. 1. Flow of the game theoretic scheduling process. Bidding and the Nash equilibrium are applied level by level until all operations are scheduled.

V. POWER-REDUCTION TECHNIQUES (GAME-THEORETIC EXTENSIONS)

- 1) **Functional unit sharing:** reuse the same unit for the same type of operations $\Rightarrow$ less switching $\Rightarrow$ less power.
- 2) **Path balancing:** if inputs A and B of an adder arrive at different times, the unit may compute twice (extra switching). Delay the early signal so both inputs arrive together.
- 3) **Register assignment:** assign the right register so signals arrive on time, reducing glitching and power; wrong assignment increases glitching and waste.

These three techniques operate inside the game-theoretic framework to make scheduling and binding more power efficient.

VI. ASSUMPTIONS AND SCOPE

- **Objective:** minimise average power; latency is *not* the primary optimisation target.
- **Resource constraint:** a limited number of functional units per type (e.g. 2–3) per control cycle.
- **First-price sealed-bid auction:** bids (power costs) are fixed once placed; the lowest wins.
- **Tie-breaking:** if two units place the same lowest bid, the choice falls back to other conditions, or is random if all else is equal.
- **Interconnect binding:** not yet included in the base approach (left for future work).

VII. RUNTIME AND COMPLEXITY (BRIEFING NOTES)

- Nash equilibrium for an N -player game with S strategies each is $\mathcal{O}(N^S NS)$ [1].
- Because only a small number of units of each type exist (2–3 players per game), runtimes stay small in practice.

- The Nash equilibrium is applied *multiple times* per DFG — once per level (scheduling) or per control step (binding).
- Payoff-matrix construction is polynomial: $\mathcal{O}(MO)$ in number of modules M and operations O .
- **Caveat:** Nash equilibrium is NP-complete in general; large systems can become slow.

VIII. RELEVANCE FOR EMBEDDED SYSTEMS

- Directly targets power $\Rightarrow$ longer battery life, less heating, maintained reliability.
- No area increase and comparable runtime to older methods (only a small latency cost).
- Especially useful for IoT, smartwatches, mobile chips, sensors, and AI accelerators with small batteries and strict power budgets.

REFERENCES

- [1] A. K. Murugavel and N. Ranganathan, "A game theoretic approach for power optimization during behavioral synthesis," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 11, no. 6, pp. 1031–1043, Dec. 2003.
- [2] W. T. Shiue and C. Chakrabarti, "ILP-based scheme for low-power scheduling and resource binding," in *Proc. Int. Symp. Circuits and Systems*, vol. 3, 2000, pp. 279–282.
- [3] C. Small, "Shrinking devices put the squeeze on system packaging," *Electron. Design News*, vol. 39, pp. 41–46, Feb. 1994.
- [4] L. Benini and G. De Micheli, "System-level power optimization: Techniques and tools," *ACM Trans. Design Automation of Electron. Syst.*, vol. 5, no. 2, pp. 115–192, Apr. 2000.
- [5] D. Gajski, N. Dutt, A. Wu, and Y.-L. Lin, *High-Level Synthesis: Introduction to Chip and System Design*. Norwell, MA: Kluwer, 1992.
- [6] P. G. Paulin and J. P. Knight, "Scheduling and binding algorithms for high-level synthesis," in *Proc. Design Automation Conf.*, 1989, pp. 1–6.
- [7] S. P. Mohanty, N. Ranganathan, and S. K. Chappidi, "Simultaneous peak and average power minimization during datapath scheduling for DSP processors," in *Proc. Great Lakes Symp. VLSI*, 2003, pp. 215–220.
- [8] A. Dasgupta and R. Karri, "Simultaneous scheduling and binding for power minimization during micro-architecture synthesis," in *Proc. Int. Symp. Low-Power Electronics Design*, 1995, pp. 69–74.
- [9] J. F. Nash, "Non-cooperative games," *Ann. Math.*, vol. 54, no. 2, pp. 286–295, 1951.